Novo-Blog

Comprehensive lightweight blogging platform prototype built with a TypeScript Node backend and a React + Vite frontend.

Table of contents

- Project overview
- Features
- Architecture and folder structure
- Tech stack
- Setup & running locally
- Environment variables
- Code explanation (backend & frontend)
- Typical API routes
- Screenshots / outputs
- Development workflow
- Deployment notes
- Contributing
- License & contact

Project overview

Novo-Blog is a minimal full-stack blog application intended as a demonstration and starter template for building CRUD apps with authentication. The repo separates concerns between a TypeScript/Express backend (API + DB models) and a React + Vite frontend (UI, routing, and API integration).

Primary goals:

- Demonstrate secure user signup/signin with JWT authentication.
- Provide CRUD endpoints for blog posts (create, read, update, delete).
- Show a modern React + TypeScript frontend with routing, reusable components, and API integration.

Features

- User signup and signin (JWT)
- Create, view, list, edit, and delete blog posts
- Protected routes for publishing and editing
- Simple responsive UI with reusable components

Architecture and folder structure

Top-level layout:

- `backend/` — Express + TypeScript API
- `frontend/` — React + TypeScript + Vite client

- `comman/` — shared/common code (if used across services)

Key backend files:

- `backend/src/server.ts` — Express application entry, middleware, and route mounting
- `backend/src/db.ts` — database connection and helpers
- `backend/src/models/Post.ts` — post model and schema
- `backend/src/models/User.ts` — user model and schema
- `backend/src/routes/*` — route handlers for auth and posts

Key frontend files:

- `frontend/src/main.tsx` — React app bootstrap and routing
- `frontend/src/config.ts` — client-side configuration (e.g., `BACKEND_URL`)
- `frontend/src/pages/*` — page-level views (Blogs, Blog, Publish, Signin, Signup)
- `frontend/src/components/*` — UI components (Auth, BlogCard, FullBlog, etc.)
- `frontend/src/hooks/*` — shared hooks (current user, API helpers)

Tech stack

- Frontend: React, TypeScript, Vite, React Router, Axios (or fetch)
- Backend: Node.js, TypeScript, Express, Mongoose (MongoDB)
- Database: MongoDB (Atlas or local instance)
- Auth: JWTs, password hashing with `bcryptjs`

Setup & running locally

Prerequisites:

- Node.js (v16+ recommended)
- npm (or pnpm/yarn)
- MongoDB (local or Atlas cluster)

1. Clone the repository

```
git clone <repo-url>
cd Novo-Blog
```

2. Backend setup

```
cd backend
npm install
```

Create a `.env` file in `backend/` with the environment variables listed below.

Run the backend in development mode:

```
npm run dev
```

The server uses `ts-node-dev` for fast TypeScript reloads. The default port is `4000` (configurable via `PORT`).

3. Frontend setup

```
cd frontend
npm install
npm run dev
```

By default the frontend expects the backend at `http://localhost:4000`. Confirm or update `frontend/src/config.ts` or the `VITE_BACKEND_URL` env var.

Environment variables

Create `backend/.env` with at least:

```
MONGODB_URI=<your-mongodb-connection-string>
JWT_SECRET=<strong-jwt-secret>
PORT=4000
```

Optional:

- `NODE_ENV=development|production`

If using MongoDB Atlas, use your cluster connection string. For local MongoDB, a typical `MONGODB_URI` might be `mongodb://localhost:27017/novoblog`.

Code explanation

Backend

- `server.ts` — creates an Express app, applies JSON/body parsers, CORS, logging middleware, and mounts API routes. It imports the DB connector then starts listening on `PORT`.
- `db.ts` — central place to connect to MongoDB using Mongoose. It exports the connection and helper functions used by models and routes.
- `models/User.ts` — Mongoose schema for users, includes fields like `email`, `passwordHash`, `createdAt`. Password hashing should be applied during signup (bcryptjs) and never stored in plain text.
- `models/Post.ts` — schema containing `title`, `body`, `author` (ObjectId ref to `User`), `createdAt`, and `updatedAt`.
- `routes/` — route handlers for API endpoints (auth and posts). Typical endpoints:
  - `POST /api/v1/user/signup` — create user, hash password, return `{ token }`.
  - `POST /api/v1/user/signin` — validate credentials and return `{ token }`.
  - `GET /api/v1/posts` — list posts (public)
  - `GET /api/v1/posts/:id` — get a single post

    - `POST /api/v1/posts` — create post (protected)
    - `PUT /api/v1/posts/:id` — update post (protected/author-only)
    - `DELETE /api/v1/posts/:id` — delete post (protected/author-only)

Frontend

- `main.tsx` — mounts React app and Router. Wraps app with any providers (Auth context).
- `config.ts` — contains `BACKEND_URL` (derived from `VITE_BACKEND_URL` environment variable).
- `pages/` — page-level components that fetch data from the backend and render content.
- `components/Auth.tsx` — signin/signup form that posts credentials and handles token storage.
- `hooks/useCurrentUser.ts` — fetches and caches authenticated user details using the stored JWT.

Authentication flow

- On signup/signin, backend returns JSON `{ token }` (JWT). Frontend stores this token (note: for production consider `HttpOnly` cookie instead of `localStorage`).
- Frontend attaches `Authorization: Bearer <token>` to protected API requests.

Typical API usage examples

Fetch posts (public):

```
fetch(`${BACKEND_URL}/api/v1/posts`).then(r => r.json())
```

Sign in (example):

```
fetch(`${BACKEND_URL}/api/v1/user/signin`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password })
}).then(r => r.json())
```

Screenshots / outputs

Place screenshots demonstrating the UI under `frontend/public/screenshots` and reference them in this README. Example file names:

- `frontend/public/screenshots/home.png`
- `frontend/public/screenshots/publish.png`
- `frontend/public/screenshots/fullpost.png`

Add the images and then embed them in this README using standard Markdown image links.

Development workflow

- Start backend with `npm run dev` (from `backend/`) and frontend with `npm run dev` (from `frontend/`).

- For backend changes that affect types, tests, or production build, run `npm run build` to compile TypeScript.
- Use feature branches and PRs for changes. Keep commits small and focused.

Deployment notes

- Backend: build a production bundle (`tsc` build) and deploy to Node host (DigitalOcean, Heroku, Fly, AWS, etc.) or containerize with Docker.
- Frontend: build static assets with `npm run build` and deploy to Netlify, Vercel, or any static host.
- Use environment variables in your host/platform settings (MongoDB connection, JWT secret).

Contributing

- Fork the repo, create a feature branch, and open a pull request with tests or screenshots showing your changes.
- Suggested first tasks: add `.env.example`, add seed script to populate test users and posts, improve UI styling, add pagination and search.

License & contact

This project is provided as-is for educational/demo purposes. Add a license file (e.g., MIT) if you want to open-source it.

For questions or help, open an issue in this repository.

If you'd like, I can also:

- Insert code snippets from `backend/src` and `frontend/src` directly into the **Code explanation** section.
- Add a `.env.example` and a simple seed script under `backend/scripts/seed.ts`.
- Embed screenshots if you upload or commit them to `frontend/public/screenshots`.